

Compound data types:1.String**2.Tuples.****3.List.****4.Dictionaries.****1. String: It is an immutable data type. They are in quotes (' , " , [] or in triple quotes).****i. Concatenation: We use to merge the different string.**

```
g="Hello",n=" World"
```

```
r=g+n
```

```
print(r) # Hello World
```

ii. String length: This function will count all the characters including spaces and symbols.

```
r="Hello World."
```

```
print(len(r)) #12
```

iii. lc: It convert all strings to lower case strings.

```
def to_lowercase(s):
```

```
    return s.lower()
```

```
s="HeLLo woRLD!"
```

```
print("In lowecase:",to_lowercase(s)) # In lowecase: hello world!
```

iv. uc: It convert all strings to upper case strings.

```
def to_uppercase(s):
```

```
    return s.upper()
```

```
s="HeLLo woRLD!"
```

```
print("In uppercase:",to_uppercase(s)) # In uppercase: HELLO WORLD!
```

v. strip: It will eliminates the spaces in the string from the start or the end.

```
def to_strip(s):
```

```
    return s.strip()
```

```
s=" Nowadays "
```

```
print("In Stripped:",to_strip(s)) #Nowadays
```

vi. replace: This function helps to replace any word with another word in the string.

```
def to_replace(s,old,new):  
    return s.replace(old,new)  
  
s="Nice Guy."  
old="Nice"  
new="Bad"  
  
print("New String:",to_replace(s,old,new)) #New String: Bad Guy.
```

vii. count: It will count the same words in the string.

```
s="Hello!,Hello is anyone there?"  
  
print(s.count("Hello")) #2
```

viii. title: It will convert all lowercase alphabet to upper case alphabet(only starting one of each words).

```
t="we are here"  
  
print(t.title()) #We Are Here
```

ix. swapcase: It will convert lower to upper or vice versa.

```
t="The PArTy iS WAitiNG"  
  
print(t.swapcase()) #tHE paRTy Is waiting
```

x. center: It will shift the sentence to the given centered position.

```
t="YESSSS"  
  
print(t.center(24,"*")) #  
  
*****YESSSS*****
```

xi. split: It splits the string into different words.

```
def to_split(s):  
    return s.split()  
  
s="We did it!"  
  
print(to_split(s)) #['We', 'did', 'it!']
```

xii. join: It joins the split word to a string.

```
def to_join(s):  
    return "".join(s)  
  
s=["We ","did ","it! "]  
  
print(to_join(s)) #We did it!
```

xiii. multiline string: We can use three double quotes for multiple lines.

```
m="""
Believe it!

Idiot

Do your best.
"""

print(m) # Believe it!

        Idiot

        Do your best.
```

xvi. “==” = We can use this function to compare between two string. It will give o/p in logical terms.

```
m=12
n=23

print(m==n) #False
```

xvii. palindrome:

```
def is_palindrome(m):
    return m==m[::-1]

m="Tomato"

print("Plaindrome:",is_palindrome(m)) # Plaindrome: False
```

xv. reverse string:

```
def is_reverse(m):
    return m[::-1]

m="Tomato"

print("Reverse:",is_reverse(m)) # Reverse: otamoT
```

2. Lists: A list is a data structure in python that is a mutable , ordered sequence of element.

List are defined by square brackets []. They can hold multiple data type.

i. insert(): It is used to insert element at any index specified by the user.

```
a=["apple","pears","rolls","litchi","banana"]  
a.insert(0,"mango")  
print(a)  #['mango', 'apple', 'pears', 'rolls', 'litchi', 'banana']
```

ii. extend(): It is used to add elements of an iterable (string,list,tuple.set.etc) to the end of another list.

```
a=["apple","pears","rolls","litchi","banana"]  
b=["tomato","potato","carrot"]  
a.extend(b)  
print(a)  #['apple', 'pears', 'rolls', 'litchi', 'banana', 'tomato', 'potato', 'carrot']
```

iii. remove(): It removes specified element from the string. It operates on the elements value, not its index.

```
a=["apple","pears","rolls","litchi","banana"]  
a.remove("banana")  
print(a)  #['apple', 'pears', 'rolls', 'litchi']
```

iv. pop(): This method removes the last element form the list by default. We can also specifies the position in the list by passing the index of the element.

```
a=["apple","pears","rolls","litchi","banana"]  
a.pop(3)  
print(a)  #['apple', 'pears', 'rolls', 'banana']
```

v. del(): The del keyword is used to delete object since in python everything is the object.

```
a=["apple","pears","rolls","litchi","banana"]  
del a[2]  
print(a)  #['apple', 'pears', 'litchi', 'banana']
```

vi. clear(): Removes all the element form the list. This does not return any value.

```
a=["apple","pears","rolls","litchi","banana"]  
a.clear()  
print(a)  #[]
```

vii. sort(): This is used to sort elements in ascending order.

```
a=["apple","pears","rolls","litchi","banana"]
```

```
a.sort()

print(a) #['apple', 'banana', 'litchi', 'pears', 'rolls']
```

viii. copy(): It returns a shallow copy of the list.

```
a=["apple","cake","rolls"]

b=a.copy()

print(b) #['apple', 'cake', 'rolls']
```

ix. concatenation(): This function helps to merge two or more list which gives the new string from the original string.

```
l1=[1,3,5]

l2=[2,4,6]

print(l1+l2) #[1, 3, 5, 2, 4, 6]
```

x. append(): This is a function used to add an item to the end of the list.

```
a=["apple","cherry","mango"]

a.append("pear")

print(a) #['apple', 'cherry', 'mango', 'pear']
```

3. Dictionaries: Dictionaries are mutable data structure that allow you to store key value pair. This is created by using curly braces {}. It accepts multiple data type.

i. Add item: One value at a time can be added to a dictionary by defining value along with the key.

```
country_capital={"england":"london","canada":"ottawa","germany":"berlin"}

country_capital["italy"]="rome"

print(country_capital) #{'england': 'london', 'canada': 'ottawa', 'germany': 'berlin', 'italy': 'rome'}
```

ii. Remove: We can use del function to remove particular value by defining the key.

```
country_capital={"england":"london","canada":"ottawa","germany":"berlin"}

del country_capital["germany"]

print(country_capital) #{'england': 'london', 'canada': 'ottawa'}
```

iii. Access the value of key: We can access the value by specifying the key value.

```
country_capital={"england":"london","canada":"ottawa","germany":"berlin"}

print(country_capital["england"]) #london
```

4. Tuple: They are written with round brackets () and they are separated by using comma “,”. We can make tuple of one element, but there must be a trailing comma to make it a tuple. We can't add/remove item once tuple is created. It allows duplicate values. We can find item in a tuple.

Characteristics:

- 1.They are in ordered sequence.**
- 2.They are light weight.**
- 3.Indexable.**
- 4.Immutable.**
- 5.Heterogeneous. [Multiple data type]**
- 6.Nestable.**
- 7.Iterable.**
- 8.Sliceable.**
- 9.Combinable/Concatenate. [+]**
- 10.Appending/Extending is not possible.**

i. len(): This function is used to return the number of elements in a tuple. This returns an integer value representing the length of the tuple.

Ex:

```
my_tuple = (1, 2, 3, 4, 5)
print(len(my_tuple)) #5
```

ii. tuple(): This function is used to create a tuple from a list, set, or any iterable object.

Ex:

```
my_list = [1, 2, 3, 4, 5]
my_tuple = tuple(my_list)
print(my_tuple) # (1, 2, 3, 4, 5)
```

iii. count(): It returns the number of times a specified element appears in a tuple.

Ex:

```
my_tuple = (1, 2, 3, 4, 3, 5, 3)
count = my_tuple.count(3)
print(count) #3
```

iv. max(): The max() function is used to return the maximum value in a tuple. It takes a tuple as an argument and returns the maximum value in the tuple.

Ex:

```
my_tuple = (5, 6, 7, 8, 9)
```

```
print(max(my_tuple)) #9
```

v. min(): The min() function is used to return the minimum value in a tuple. It takes a tuple as an argument and returns the minimum value in the tuple.

Ex:

```
my_tuple = (5, 6, 7, 8, 9)
```

```
print(min(my_tuple)) #5
```

Data types:

1. Numeric: The numeric data type in Python represents the data that has a numeric value.

i) Integer: This value is represented by int class. It contains positive or negative whole numbers

ii) Float: The float data types are used to store positive and negative numbers with a decimal point

iii) Complex: The complex data type in python consists of two values, the first one is the real part of the complex number, and the second one is the imaginary part of the complex number.

2. Dictionary: Dictionaries are used to store data values in key:value pairs.

A dictionary is a collection which is ordered, changeable and do not allow duplicates.

3. Boolean: In Python, boolean variables are defined by the True and False keywords.

4. Set: Sets are used to store multiple items in a single variable. A set is a collection which is *unordered*, *unchangeable**, and *unindexed*.

5. Sequence: Sequence Data Types are used to store data in containers in the [Python computer language](#). The different types of containers used to store the data are [List](#), [Tuple](#), and [String](#).

i)Strings.

ii)List.

iii)Tuple.

Working with functions:

Function parameter: A parameter is the variable listed inside the parentheses in the function definition. An argument is the value that is sent to the function when it is called.

Ex:

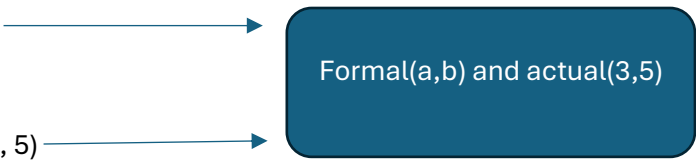
```
def greet(name):  
    print("Hello", name)  
  
greet("John") #Hello John
```

Formal parameter: They are also known as formal arguments. They are defined in function declaration. They represent the data that the function expects to receive when called. They are input data.

Actual parameter: They are also known as actual arguments. They correspond to the formal parameters in the function's definition and supply the necessary input data for the function to execute. Actual parameters can be constants, variables, expressions, or even function calls.

Ex:

```
def sum_numbers(a, b):  
    return a + b  
  
result = sum_numbers(3, 5)  
  
print("Sum:", result) #8
```



The diagram illustrates the relationship between formal parameters and actual arguments. A blue rounded rectangle contains the text "Formal(a,b) and actual(3,5)". Two blue arrows point from the code to this box: one from the parameters "(a, b)" in the function definition "def sum_numbers(a, b):" and another from the arguments "(3, 5)" in the function call "sum_numbers(3, 5)".

Global variable: Global variables are those which are not defined inside any function.

Ex:

```
message="Hello"  
  
def greet():  
    print("Global",message)  
  
greet() # Global Hello
```

Local variable: Local variables are those which are defined inside a function.

Ex:

```
def greet():  
    message="Hello"  
    print("Local",message)  
  
greet() # Local Hello
```


Range function(start, stop, step): The Python range() function returns a sequence of numbers, in a given range. The most common use of it is to iterate sequences on a sequence of numbers using Python loops.

Ex:

```
x = range(6)
```

```
for n in x:
```

```
    print(n) #0
```

```
            1
```

```
            2
```

```
            3
```

```
            4
```

```
            5
```

Value Returning and Non-value Returning Function:

1.The Python [return statement](#) is a special statement that you can use inside a [function](#) or [method](#) to send the function's result back to the caller. A return statement consists of the [return keyword](#) followed by an optional return value.

Ex:

```
def calculate_area(length, width):
```

```
    area = length * width
```

```
    return area
```

```
area = calculate_area(50, 50)
```

```
print("Calculated area:",area) #Calculated area:2500
```

2.In python it is possible to compose a function without a return statement. This function is called void, and they return "None", python's special object for "nothing".

Ex:

```
def greet(who):
```

```
    print("Hello",who,"!")
```

```
    print("What a lovely day")
```

```
greet("Rohit") # Hello Rohit !
```

```
                What a lovely day
```

Keyword And Default arguments anonymous function:

1.Keyword arguments (or named arguments) are values that, when passed into a function, are identifiable by specific parameter names. A keyword argument is preceded by a parameter and the assignment operator, =.

Ex:

```
def my_function(child3, child2, child1):  
    print("The youngest child is " + child3)  
  
my_function(child1 = "Rohit", child2 = "Aditiya", child3 = "Nikhil")  
  
# The youngest child is Nikhil
```

2. In Python, we can provide default values to function arguments. We use the = operator to provide default values.

Ex:

```
def add_numbers( a = 7, b = 8):  
    sum = a + b  
    print('Sum:', sum)  
  
add_numbers(2,3)  
add_numbers(a=5)  
add_numbers()  
  
# Sum: 5  
Sum: 13  
Sum: 15
```

Lambda Anonymous Function:

Python Lambda Functions are anonymous functions means that the function is without a name. As we already know the *def* keyword is used to define a normal function in Python. Similarly, the *lambda* keyword is used to define an anonymous function.

Ex:

```
sum_numbers=lambda x,y:x+y  
result=sum_numbers(5,3)  
print(result) #8
```

List Comprehension: List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

Ex:

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
```

```
newlist = [x for x in fruits if "a" in x]
```

```
print(newlist) # ['apple', 'banana', 'mango']
```

Comparison between String, List and Tuple:

○ STRING	○ LIST	○ TUPLE
○ Immutable	○ Mutable	○ Immutable
○ Ordered/Indexed	○ Ordered/Indexed	○ Ordered/Indexed
○ Allows Duplicate Values	○ Allows Duplicate Values	○ Allows Duplicate Values
○ Empty string = ""	○ Empty list = []	○ Empty tuple = ()
○ String with single element = " A "	○ List with single item = [" Apple "]	○ Tuple with single item = (" Apple ",)
○ Stores characters or strings	○ It can store any data type str, list, set, tuple, int and dictionary	○ It can store any data type str, list, set, tuple, int and dictionary

Introduction of Python.**History of programming languages in India:****Python programming language introduced in India:**

Python was introduced in India in the early 2000s and quickly gained popularity because of simplicity and versatility. The language became a preferred choice for educational institutions and the IT industry, which helps in tech sectors. Today, Python is widely taught in schools and colleges and is heavily used in various industries including AI, data science, and web development.

List of its different version with its pros:

1. Python's journey began with version 0.9.0 released by Guido van Rossum in 1991. Initial release includes string, list, dictionary and other modules.
2. Python 1.0 marked the official public release which includes Lambda, Map, Filter, Reduce and Exception handling.
3. Python 1.5 (1997) which includes Standard Library Enhancements and Unicode Support.
4. Python 2.0 (2000) which includes List comprehension, Garbage collection and Unicode support.
5. Python 2.7 (2010) which includes ordered dictionaries, set literals and improved syntax.

Programming language that influenced python:

1. Python is influenced by other languages such as ABC, C, C++, Icon, Java, Perl, Ada, Standard ML, etc.
2. Python is actually based directly on a language called ABC - which also has the unusual feature of having block structure determined by indentation.

Contributions in various fields:

Python is used in business applications for a wide range of purposes, including data analysis, automation, and web development which interact with business databases and APIs.

Ways python developed in business aspects:

1. Python Encourages Faster Project Development.
2. It Provides an Ideal Platform for Web Development.
3. Develop Portable Codes Using Python.
4. Availability of Multiple Frameworks for Better UI Experience.
5. Best Community Support System.